

APUNTES MAESTROS CON EJEMPLOS

ECOSISTEMA DE DESARROLLO FRONTEND • "ACME SCHOOL"

1. Almacenamiento y Estado (LocalStorage y JSON)

Concepto: `LocalStorage` solo almacena texto plano (strings). Para guardar objetos o arrays de JavaScript, es obligatorio convertirlos a texto mediante `JSON.stringify()`. Al recuperarlos, se utiliza `JSON.parse()` para reconstruir la estructura de objeto original y poder manipular sus propiedades.

Ejemplo práctico para el cuaderno:

```
// 1. Convertir y Guardar (JSON.stringify)
const estudiante = { nombre: "Carlos", nota: 10 };
localStorage.setItem('sesion', JSON.stringify(estudiante));

// 2. Leer y Reconstruir (JSON.parse)
const datosGuardados = localStorage.getItem('sesion');
const estudianteRecuperado = JSON.parse(datosGuardados);

console.log(estudianteRecuperado.nombre); // Imprime: "Carlos"
```

2. Manipulación del DOM (app.js)

Concepto A: Métodos Estáticos (`static`)

Permiten invocar una función directamente desde la clase constructora sin necesidad de instanciar o crear un objeto nuevo utilizando la palabra clave `new`. Es ideal para funciones utilitarias o helpers.

```
class Calculadora {
  static sumar(a, b) {
    return a + b;
  }
}

// ¡No necesito hacer "new Calculadora()"!
console.log(Calculadora.sumar(5, 5)); // Imprime 10
```

Concepto B: Delegación de Eventos

En lugar de asignar un escuchador de eventos (listener) a cada botón individual (por ejemplo, botones de eliminar en una lista dinámica), se añade un único evento al contenedor padre. Luego se evalúa la propiedad `e.target` con el método `matches()` para identificar exactamente sobre qué elemento hijo se hizo clic.

```
// Le pongo el evento al contenedor padre entero (la tabla)
const tabla = document.getElementById('exámenes-tabla');

tabla.addEventListener('click', (e) => {
  // Si le di clic exactamente a algo con la clase "btn-eliminar"
  if (e.target.matches('.btn-eliminar')) {
    console.log("Borrando examen...");
  }
});
```

3. Enrutamiento y Sesión

Concepto A: Extraer parámetros de la URL (`URLSearchParams`)

Herramienta nativa para desglosar y leer las variables enviadas por la barra de direcciones (Query Strings). Por ejemplo, si un alumno ingresa a la dirección `examen.html?id=456`, podemos aislar el identificador fácilmente.

```
// Leemos la URL actual (Simulando window.location.search)
const url = new URLSearchParams("?id=456");

// Extraemos el valor asociado a la clave "id"
const idExamen = url.get("id");
console.log(idExamen); // Imprime: "456"
```

Concepto B: Intercepción de Formularios (`e.preventDefault()`)

Cancela el comportamiento por defecto de un evento. En el caso de los formularios, evita que el navegador recargue la página o parpadee al presionar el botón de envío, cediendo el control total del flujo de datos a JavaScript.

```
const formulario = document.querySelector('form');

formulario.addEventListener('submit', (e) => {
  e.preventDefault(); // Detiene la recarga por defecto
  console.log("¡Formulario atrapado! Ahora decido yo a dónde ir.");
});
```

4. Web Components y Shadow DOM

Concepto: Permite crear etiquetas HTML personalizadas y reutilizables (ej. `<resolver-examen>`). Al encapsularlas con `Shadow DOM`, se genera una burbuja de aislamiento absoluto que previene que los estilos globales de la aplicación afecten al componente o que scripts externos alteren su estructura interna (esencial para evitar trampas).

Ejemplo práctico para el cuaderno:

```
// 1. Heredamos de HTMLElement de manera nativa
class TarjetaExamen extends HTMLElement {
  constructor() {
    super();

    // 2. Creamos la burbuja protectora (Shadow DOM en modo abierto)
    const shadow = this.attachShadow({ mode: "open" });

    // 3. Inyectamos nuestro código y estilos encapsulados
    shadow.innerHTML = `
      <style>
        h1 { color: red; }
      </style>
      <h1>Soy un Web Component blindado</h1>
    `;
  }
}

// 4. Registramos la nueva etiqueta en el navegador
customElements.define("tarjeta-examen", TarjetaExamen);

// En el HTML ahora puedes escribir directamente: <tarjeta-examen></tarjeta-examen>
```

5. Lógica Anti-Trampas y Evaluación

Concepto A: Temporizador a prueba de recargas

Los contadores basados únicamente en un `setInterval` tradicional pierden el progreso si el usuario refresca la página. Para blindarlo, se registra la marca de tiempo absoluta de inicio (timestamp) y se calcula la diferencia matemática contra la hora real de cada segundo.

```
// Guardamos el momento exacto (en milisegundos) en que inició el examen
const horaInicio = new Date().getTime();

const cronometro = setInterval(() => {
  // Capturamos el tiempo exacto en ESTE SEGUNDO
  const horaActual = new Date().getTime();

  // La diferencia matemática es inalterable, aunque recargues la página
  const tiempoTranscurrido = horaActual - horaInicio;
  console.log("Han pasado " + tiempoTranscurrido + " milisegundos");
}, 1000);
```

Concepto B: Recolectar respuestas seleccionadas

Uso de selectores avanzados del DOM para extraer de forma limpia y directa los datos del examen, filtrando únicamente aquellos elementos interactivos que el usuario ha marcado activamente.

```
// Busca todos los inputs tipo radio, pero SOLO los que están marcados (:checked)
const respuestasMarcadas = document.querySelectorAll('input[type="radio"]:checked');

respuestasMarcadas.forEach(input => {
  console.log("El alumno seleccionó la opción: " + input.value);
});
```

6. Arquitectura Visual y CSS

Concepto: Inyección dinámica de clases en el árbol HTML basándose en el estado de la lógica del negocio (ej. si el alumno aprobó o no). Se utilizan Template Literals (comillas invertidas ```) combinados con operadores ternarios para agilizar y limpiar el código.

Ejemplo práctico para el cuaderno:

```
const notaAlumno = 8;
const notaMinima = 6;

// Operador ternario: Condición ? Caso Verdadero : Caso Falso
const claseCss = notaAlumno >= notaMinima ? 'status-aprobado' : 'status-reprobado';

// Inyectamos dinámicamente la clase y el valor dentro del template HTML
const resultadoHTML = `<div class="${claseCss}">Tu nota es ${notaAlumno}</div>`;

// Insertamos de manera eficiente el elemento en el DOM
document.body.insertAdjacentHTML('beforeend', resultadoHTML);
```